



Machine Learning

BITS Pilani

Anand Narasimhamurthy



ALMLZ565, Machine Learning Support Vector Machines

Summary



- A learning technique developed by Vapnik and his team at AT&T Bell Labs.
- Motivated by the principle of **Structural Risk Minimization** (SRM)
- A **good candidate technique for high dimensional data**.
i.e. a suitable choice where the number of features is large and the available training data is relatively sparse.
- Easy to use : A **small number of hyperparameters** and hence not too much tuning required.

A few SVM applications



Face detection Osuna et al. [Osuna97]

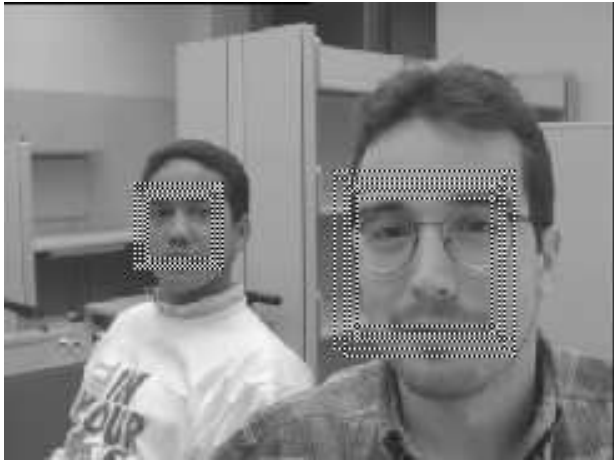
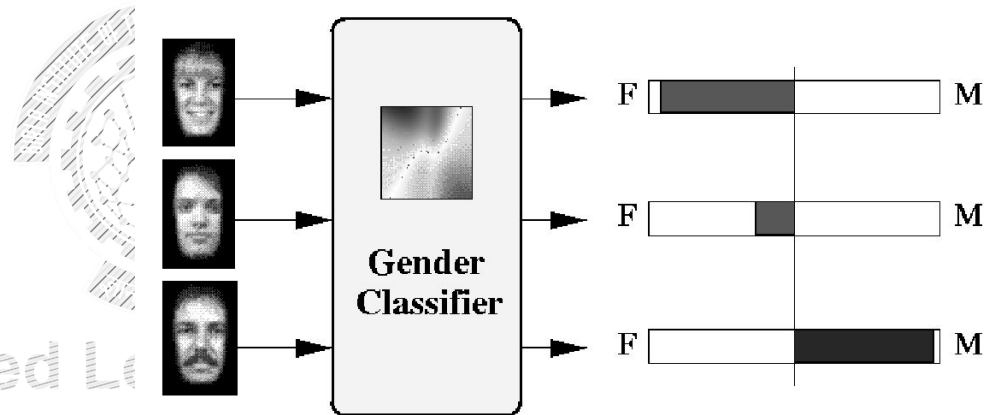


Image classification Eg.
Gender classification from
frontal face images
Mogaddham et al. [Mogaddham2002]



Other applications (for a brief overview see Chapter 8, [Christianini2000]).

- Bioinformatics, text (and hypertext) categorisation
- hand-written character recognition

Salient features of SVM



- **Maximum margin classifier.** Hence, intuitively, should yield better generalization.
- **Training an SVM** involves solution of a **Quadratic Program (QP)**.
 - The optimization problem is **convex**, hence a **global optimum**, i.e. no local minima issues
 - Although the QP is dense, efficient implementations are available.
- In an SVM classifier, the decision boundary is expressed using only a subset of training data, these data vectors are called **Support Vectors**
 - If the number of Support Vectors is small, the SVM provides a compact representation of the data and possibly additional insights.

Margin



- For a given training data (two classes) and a separating hyperplane, the **margin** refers to the distance between the convex hulls measured along a line perpendicular to the hyperplane.
- A larger margin is generally expected to provide better generalization.
- Support Vector Machine (SVM) computes the **maximum margin hyperplane** by design.
- Finding the maximum margin hyperplane can be formulated as a **Quadratic Program (QP)**

Margin

innovate

achieve

lead

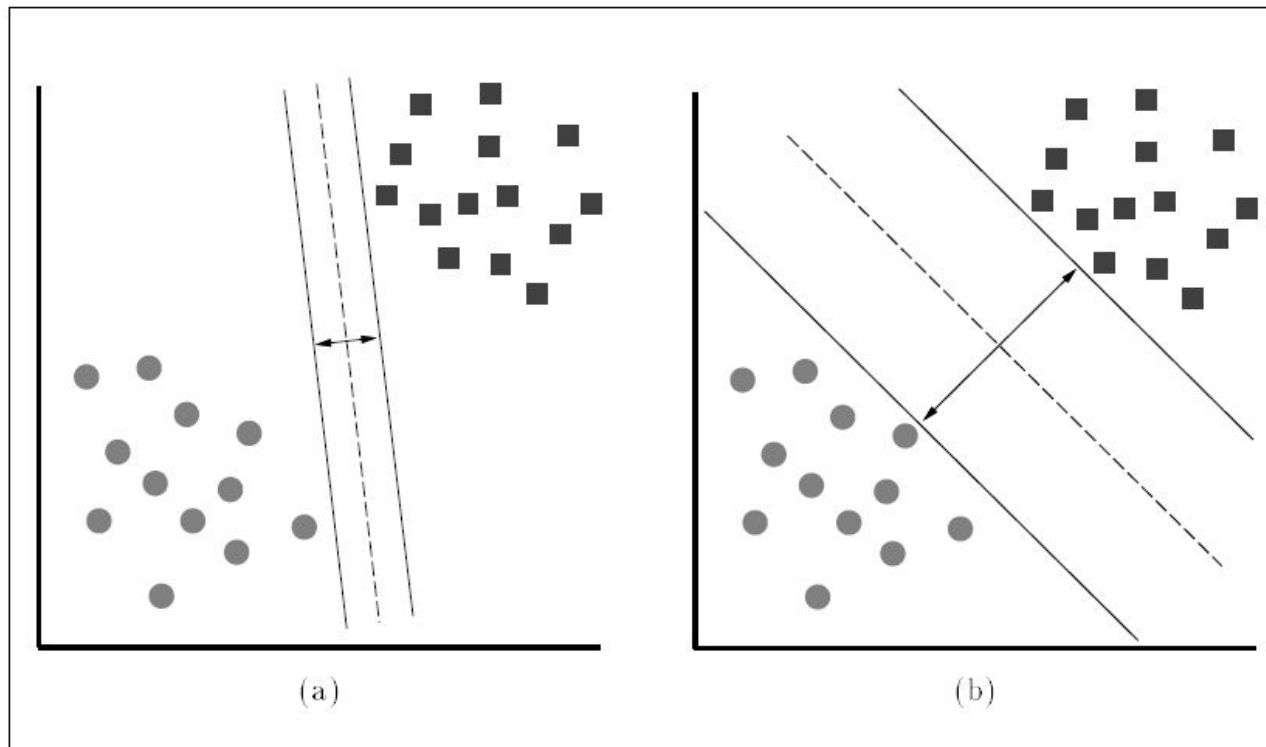


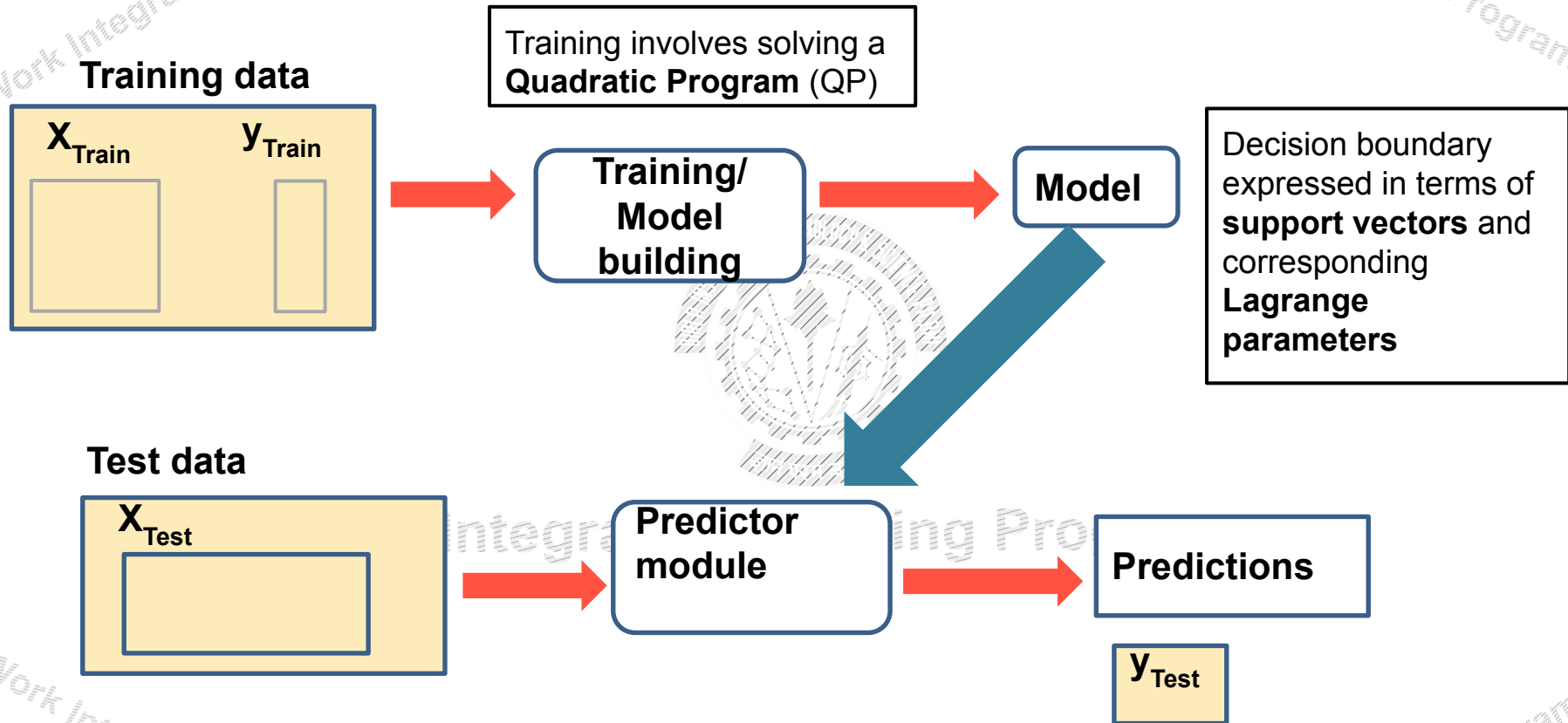
Figure 2: (a) A Separating Hyperplane with small *margin*. (b) A Separating Hyperplane with larger *margin*. A better *generalization* capability is expected from (b).

A block diagram illustrating supervised learning with SVM

innovate

achieve

lead



innovate

achieve

lead

Optimization : A general overview



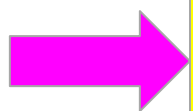
Work Integrated Learning Programmes

- Numerical optimization typically involves formulating a problem as minimization or maximization of a function of possibly multiple variables.
- Suppose there are n variables denoted as $x_1, x_2, \dots, x_n$ and let $f(x_1, x_2, \dots, x_n)$ be the function to be minimized/maximized.
 - The variables $x_1, x_2, \dots, x_n$ are referred to as **decision variables**
 - The function $f(\dots)$ is referred to as the **objective function**

Many machine learning techniques involve numerical optimization. Typically, the decision variables are **model parameters** (to be solved for) and the objective function is a **loss/cost** function

Common broad classes of optimization problems : Unconstrained optimization

Objective function
eg. $f(x_1, \dots, x_n)$ only



Unconstrained
optimization

Examples

Data fitting problems

- Linear/polynomial regression
- Ridge regression

Classification

- Logistic regression
- Neural networks

Solution techniques

- **Analytical solution using multivariate calculus :**
Directly evaluate first and second order conditions
- **Iterative methods**
Start with one or more initial solutions and converge to a better solution. Examples:
 - **Gradient descent**, Newton's method
 - Evolutionary approaches.
Eg. **Genetic Algorithms**, Simulated annealing etc.

Common broad classes of optimization problems : Constrained optimization

Objective function
 $f(x_1, \dots, x_n)$

+

**additional
constraints**

**Constrained
optimization**

A few general
solution techniques

Examples

Linear Program

Quadratic Program:
Support Vector
Machine

- Primal methods
- Penalty and barrier methods
- Dual and cutting plane methods
- Lagrangian methods

Quadratic program : General description

Quadratic Program (QP) is a constrained optimization problem where :

- The objective function is a quadratic function of decision variables
- The constraints (both equality and inequality) are also linear.
 - Equality constraints may be expressed as $A_{eq} x = b_{eq}$
 - Inequality constraints may be expressed as $A_{ineq} x \leq b_{ineq}$

where $x =$

$$\begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

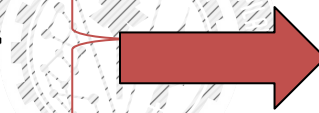
and A_{ineq} , A_{eq} , b_{ineq} , b_{eq} are matrices and vectors of compatible sizes.

A standard form of a Quadratic Program



$$\text{Let } x = [x_1, x_2, \dots, x_N]^T$$

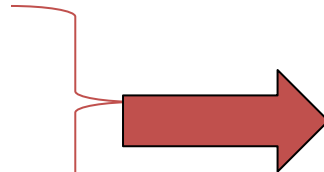
$$\min x^T H x + f^T x$$



A quadratic objective to be maximised/minimised

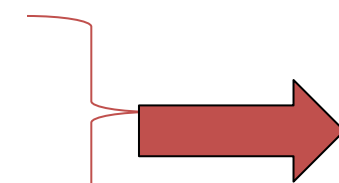
s.t.

$$Ax \leq b$$



Linear inequality constraints

$$A_{eq} x = b_{eq}$$



Linear equality constraints

A toy Quadratic Program



$$\min x_1^2 + x_2^2$$

$$\text{Here } x = [x_1, x_2]^T$$

s.t.

$$H = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \quad f = [0, 0]^T$$

$$x_1 + 2x_2 \leq 15$$

$$A = [1 \quad 2] \quad b = [15]$$

$$x_1 - x_2 = 3$$

$$A_{eq} = [1 \quad -1] \quad b_{eq} = [3]$$

Method of Lagrange multipliers

innovate

achieve

lead

Suppose the constrained optimization problem is

minimize $f(x_1, x_2, \dots, x_n)$

subject to

$$g_1(x_1, x_2, \dots, x_n) = 0$$

Constraint 1

⋮

$$g_M(x_1, x_2, \dots, x_n) = 0$$

Constraint M

Method of Lagrange multipliers

Define the Lagrangian as

$$\mathcal{L}(x_1, \dots, x_n, \lambda_1, \dots, \lambda_M) = f(x_1, \dots, x_n) - \sum_{k=1}^M \lambda_k g_k(x_1, \dots, x_n)$$

The first order condition for minimizing/maximizing the Lagrangian is

$$\nabla_{x_1, \dots, x_n, \lambda_1, \dots, \lambda_M} \mathcal{L}(x_1, \dots, x_n, \lambda_1, \dots, \lambda_M) = 0$$

$$\nabla f(\mathbf{x}) = \sum_{k=1}^M \lambda_k \nabla g_k(\mathbf{x}) \quad \Longleftrightarrow \quad \nabla f(\mathbf{x}) - \sum_{k=1}^M \lambda_k \nabla g_k(\mathbf{x}) = 0.$$

$$\Longleftrightarrow \begin{cases} \nabla f(\mathbf{x}) - \sum_{k=1}^M \lambda_k \nabla g_k(\mathbf{x}) = 0 \\ g_1(\mathbf{x}) = \dots = g_M(\mathbf{x}) = 0 \end{cases}$$

innovate

achieve

lead

SVM formulations



Work Integrated Learning Programmes

Linear classifier : a linearly separable problem

innovate

achieve

lead

Constraints

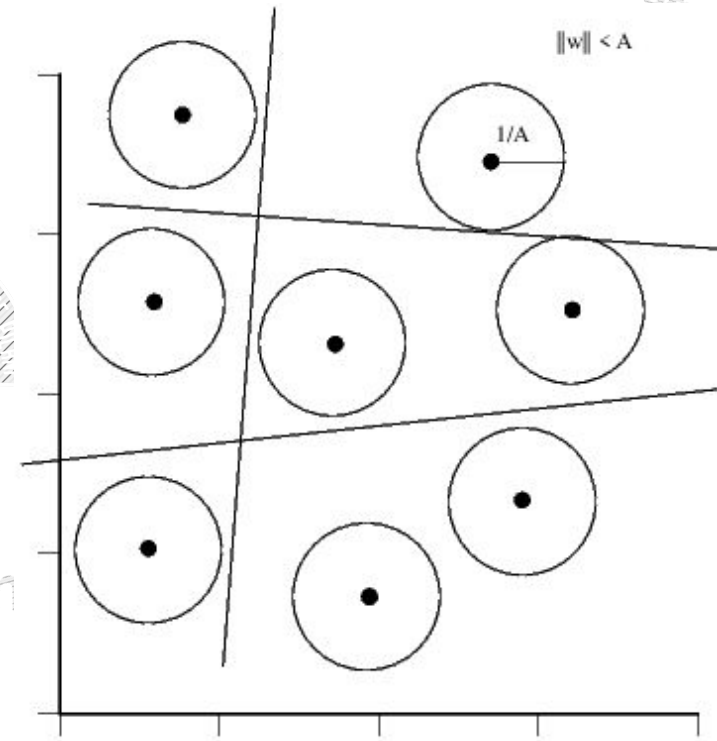
$$\begin{aligned} \mathbf{w} \cdot \mathbf{x}_i + b &\geq 1 & \forall \mathbf{x}_i \in \text{Class 1} \\ \mathbf{w} \cdot \mathbf{x}_i + b &\leq -1 & \forall \mathbf{x}_i \in \text{Class 2} \end{aligned}$$

$$f_{\mathbf{w},b} = \text{sign}(\mathbf{w} \cdot \mathbf{x} + b)$$

Hypothesis space

Bounding the norm of $\|\mathbf{w}\|$ is equivalent to constraining the spheres to remain outside spheres of radius $1/A$ centered around data points

Bounding the norm of $\|\mathbf{w}\|$ is indirectly related to maximizing the margin



SVM formulation for linearly separable case



$$\begin{aligned} \mathbf{w} \cdot \mathbf{x}_i + b &\geq 1 & \forall \mathbf{x}_i \in \text{Class 1} \\ \mathbf{w} \cdot \mathbf{x}_i + b &\leq -1 & \forall \mathbf{x}_i \in \text{Class 2} \end{aligned}$$

SVM formulation : The constraints
(one constraint for each training example)

$$\begin{aligned} &\text{Minimize} && \Phi(\mathbf{w}) = \frac{1}{2} \|\mathbf{w}\|^2 \\ &\mathbf{w}, b \\ &\text{subject to} \end{aligned}$$

$$y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1 \quad i = 1 \dots \ell$$

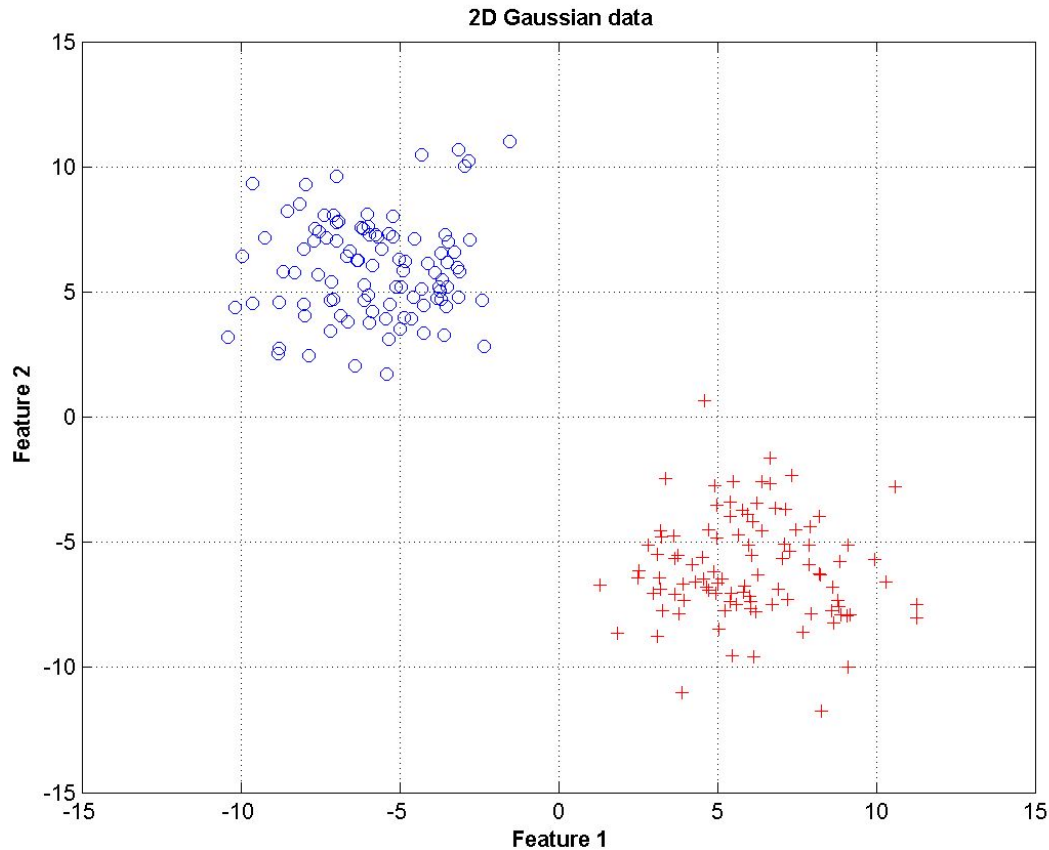
SVM : The Quadratic Program formulation

$$L(\mathbf{w}, b, \Lambda) = \frac{1}{2} \|\mathbf{w}\|^2 - \sum_{i=1}^{\ell} \lambda_i [y_i(\mathbf{w} \cdot \mathbf{x}_i + b) - 1], \quad \text{SVM formulation : The Lagrangian}$$

$$\frac{\partial L(\mathbf{w}, b, \Lambda)}{\partial \mathbf{w}} = \mathbf{w} - \sum_{i=1}^{\ell} \lambda_i y_i \mathbf{x}_i = 0$$

SVM QP formulation :
First order condition :
Derivative of the Lagrangian is 0

Illustration of SVM with a toy example with 2D data



Plot of training samples & decision boundary

innovate

achieve

lead

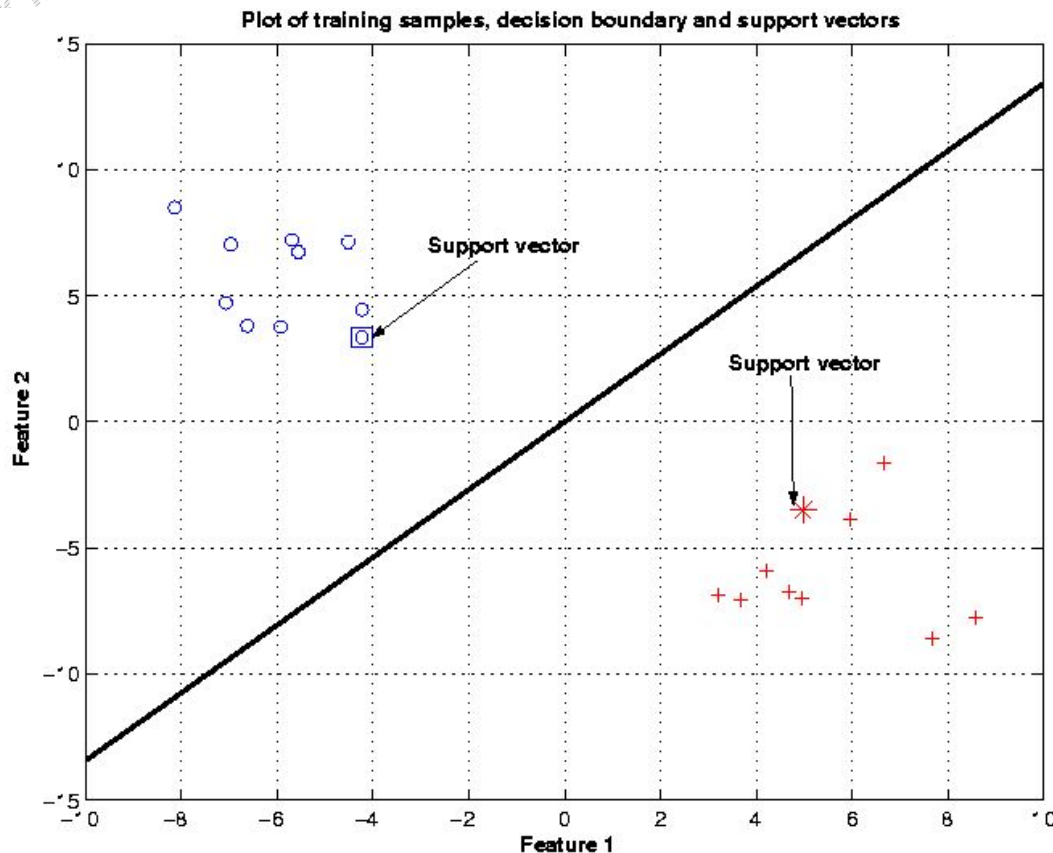


Illustration of SVM with a toy example with 2D data

innovate

achieve

lead

1	1:8.5808	2:-7.7554
1	1:7.6814	2:-8.6028
1	1:4.9446	2:-7.0276
1	1:4.9774	2:-3.5125
1	1:6.6651	2:-1.6299
1	1:5.9512	2:-3.8880
1	1:4.6761	2:-6.7830
1	1:3.6689	2:-7.0983
1	1:4.2141	2:-5.9138
1	1:3.2060	2:-6.8883
-1	1:-4.2367	2:3.3487
-1	1:-4.5217	2:7.1263
-1	1:-7.0805	2:4.7156
-1	1:-5.6921	2:7.2164
-1	1:-6.9735	2:7.0333
-1	1:-8.1428	2:8.4976
-1	1:-6.6291	2:3.8111
-1	1:-4.2309	2:4.4521
-1	1:-5.5644	2:6.7292
-1	1:-5.9375	2:3.7654

Training samples file

```
svm_type c_svc
kernel_type linear
nr_class 2
total_sv 2
rho 0.0602289
label 1 -1
nr_sv 1 1
SV
0.01515430443980411 1:4.9774 2:-3.5125
-0.01515430443980411 1:-4.2367 2:3.3487
```

The model file generated by Lib SVM.
(Linear kernel was used)

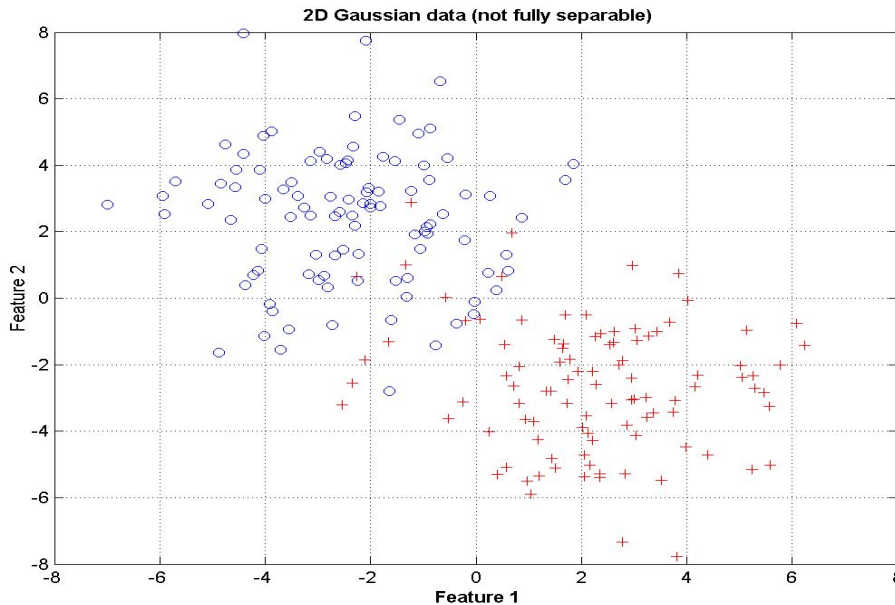
Lagrange parameters
corresponding to
the support vectors

Training samples in LibSVM format
<classLabel> <FeatureNo>:<FeatureValue> ...

Modifications of basic SVM



In most practical applications, it is too optimistic to hope for classes to be completely linearly separable



Data samples of two classes not completely separable using a linear decision boundary.

Modification in SVM formulation :

- A term (C) which penalizes misclassifications on training samples
- The hyperparameter (C) acts as a regularization term and provides trade off between accuracy on training set and generalization.

Low value of c : Lesser penalty for misclassifying training samples, more generalization capability.

High value of c : Higher penalty for misclassifying training examples, more emphasis on training accuracy

SVM formulation for linearly non-separable case



$$\text{Minimize } \Phi(\mathbf{w}, \Xi) = \frac{1}{2} \|\mathbf{w}\|^2 + C \left(\sum_{i=1}^{\ell} \xi_i \right)$$

$\mathbf{w}, b, \Xi$

subject to

$$y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1 - \xi_i$$

$$\xi_i \geq 0$$

$$i = 1, \dots, \ell$$

$$i = 1, \dots, \ell$$

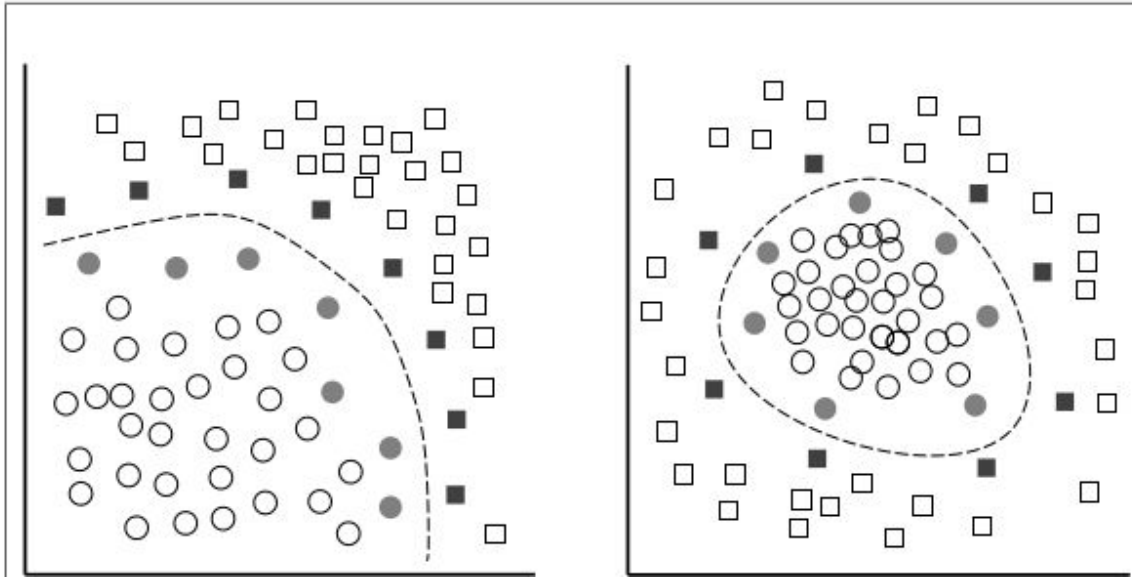


SVM formulation

Quadratic Program (Primal form)

An additional term C

Modifications of basic SVM : Non-linearly separable case



Data separable but not
by a hyperplane
i.e. the classes are not linearly
separable

Figure reproduced from Osuna et
al. [Osuna97]

Modification in SVM formulation :

- Use of kernels, enables projection of data into a space where the two classes are linearly separable.
- (possibly higher dimensional space than that in which feature vectors reside)
- The mapping function need not be explicitly known.

Kernel trick in SVM

Consider mapping a vector $\mathbf{x}$ to a vector in some high dimensional space via the mapping $\phi(\mathbf{x})$

$$\mathbf{x} \rightarrow \phi(\mathbf{x}) = (a_1\phi_1(\mathbf{x}), a_2\phi_2(\mathbf{x}), \dots, a_n\phi_n(\mathbf{x}), \dots)$$

A kernel function is one such that for two vectors $\mathbf{x}$ and $\mathbf{y}$

$$K(\mathbf{x}, \mathbf{y}) \equiv \phi(\mathbf{x}) \cdot \phi(\mathbf{y}) = \sum_{n=1}^{\infty} a_n^2 \phi_n(\mathbf{x}) \phi_n(\mathbf{y})$$

for some mapping function $\phi(\mathbf{x})$

A key property of the SV machinery is that the only quantities that one needs to compute are dot products, of the form $\phi(\mathbf{x}) \cdot \phi(\mathbf{y})$

Kernel trick in SVM

Using a kernel function $K(\cdot)$ the decision boundary can be written as :

$$f(\mathbf{x}) = \text{sign}(\phi(\mathbf{x}) \cdot \mathbf{w}^* + b^*) \Rightarrow \text{sign}\left(\sum_{i=1}^{\ell} y_i \lambda_i^* \phi(\mathbf{x}) \cdot \phi(\mathbf{x}_i) + b^*\right)$$

The decision boundary using the Lagrangian dual can be written as :

$$f(\mathbf{x}) = \text{sign}\left(\sum_{i=1}^{\ell} y_i \lambda_i^* K(\mathbf{x}, \mathbf{x}_i) + b^*\right)$$

Class label of
 i^{th} support vector

Lagrange
multiplier of
 i^{th} support vector

Commonly used kernels *



Kernel type	Kernel function
Linear	$K(x,y) = x \cdot y$
Polynomial of degree d	$K(x,y) = (a_0 + \gamma \cdot x \cdot y)^d$
Gaussian Radial Basis Function	$K(x,y) = \exp(-\gamma \ x-y\ ^2)$
Sigmoid	$K(x,y) = \tanh(\gamma \cdot x \cdot y + a_0)$

Work Integrated Learning Programmes

* The exact functions may vary depending on implementation.

References and additional links

[Christianini2000] An Introduction to Support Vector Machines and Other Kernel-based Learning Methods, Nello Cristianini John Shawe-Taylor, 2000

[Moghaddam2002] Learning Gender with Support Faces, B. Moghaddam and Ming-Hsuan Yang, IEEE Transactions on Pattern Analysis and Machine Intelligence (PAMI), Vol 24, No 7, July 2002

[Burges98] A Tutorial on Support Vector Machines for Pattern Recognition, Christopher J.C. Burges

[Osuna97] Support Vector Machines: Training and Applications, Edgar E. Osuna, Robert Freund and Federico Girosi, Technical Report, March 1997
<https://dspace.mit.edu/handle/1721.1/7290>

<https://scikit-learn.org/stable/modules/svm.html>
Scikit learn documentation on Support Vector Machines

Support Vector Machine with Scikit-Learn: A Friendly Introduction, [Riccardo Andreoni](#), October 2023

<https://medium.com/towards-data-science/support-vector-machine-with-scikit-learn-a-friendly-introduction-a2969f2ff00d>